

Rahul Traders - Application & Architecture Documentation

This comprehensive document outlines the architecture, data structures, and exact mathematical and logical calculations used to render every section of the **Rahul Traders** application.

1. Dashboard Module

The Dashboard provides a high-level overview of the company's financial and inventory health. The KPIs displayed on this screen are calculated dynamically by aggregating data across various database tables.

1.1 Total Stock Value

- **Calculation:** Iterates through all active products in the `stock_items` table.
- **Formula:** $\text{Sum}(\text{ItemQuantity} * \text{ItemRate})$ for all products belonging to the selected company.
- **Display:** Shown as a formatted currency string on the top KPI card.

1.2 Total Receivables

- **Calculation:** Fetches data from the `outstanding_receivables` table.
- **Formula:** The system checks the `closing_balance` for each invoice. If `closing_balance` is zero or null, it falls back to the original amount. $\text{Sum}(\text{closing_balance} \neq 0 ? \text{absolute}(\text{closing_balance}) : \text{absolute}(\text{amount}))$
- **Display:** Displayed as a gross total representing money owed by customers to Rahul Traders.

1.3 Total Payables

- **Calculation:** Fetches data from the `outstanding_payables` table.
- **Formula:** Similar to receivables, it aggregates `closing_balance` or amount. $\text{Sum}(\text{closing_balance} \neq 0 ? \text{absolute}(\text{closing_balance}) : \text{absolute}(\text{amount}))$
- **Display:** Displayed as a gross total representing money Rahul Traders owes to suppliers/vendors.

1.4 Entities Count

- **Customers Count:** A direct `COUNT()` of all active rows in the `customers` table.
 - **Products Count:** A direct `COUNT()` of all rows in the `stock_items` table.
-

2. Stock & Inventory Module

The Stock Screen displays the complete catalog of products available.

2.1 Item Display

- **Fields Shown:** `ItemName`, `ItemQuantity`, `ItemRate`, `StandardCost`, and `MRP`.
- **Search & Filtering:** Handled on the database level via Supabase `ilike` queries on the `ItemName` column.

2.2 Inventory Value per Item

- **Calculation:** Each list item displays its individual stock value.

- **Formula:** $\text{ItemQuantity} * \text{ItemRate}$.

3. Ledgers (Customers) Module

The Ledger Screen manages the directory of all customers and their individual financial standing.

3.1 Display Logic

- **Fields Shown:** customer_name, mobile_number, city, credit_limit, and opening_balance.
 - **Pagination:** Loads data in chunks of 1000 to maintain smooth scrolling.
-

4. Outstanding Bills (Receivables & Payables)

These screens provide aging and outstanding balance analysis.

4.1 Receivables List

- **Fields Shown:** customer_name, invoicenum, date, duedate, overdue_days, and closing_balance.
 - **Calculation:** The overdue_days are tracked to show exactly how late a payment is. The UI highlights overdue bills to assist with collection efforts.
-

5. Invoices Module (Sales & Purchase)

The Invoice screens track individual transactions and calculate taxes and discounts.

5.1 Sales Invoice Overview

- **Fields Shown:** invoice_number, invoice_date, customer_name, total_amount, and status.

5.2 Invoice Details Calculation

When viewing a specific invoice, the calculations for the line items are structured as follows: -

Subtotal before Discount: Sum of quantity * unit_price across all invoice_items. -

Discount Amount: Calculated dynamically if a discount_percentage is applied. - **GST / Taxes:** Evaluated against the gst_rate defined per item or on the gross total. - **Net Amount:** Subtotal - Discount + GST.

6. Daybook Module

The Daybook provides an audit trail of daily transactions and accounting vouchers.

6.1 Display Logic

- **Fields Shown:** date, voucher_number, voucher_type, ledger_name, and amount.
 - **Debit/Credit Parsing:** The application interprets the is_debit boolean. If true, the transaction is displayed under Debit; otherwise, under Credit.
-

7. Analytics & Reports Module

The Analytics dashboard performs complex computations using Supabase PostgreSQL Remote Procedure Calls (RPCs).

7.1 Fast Moving Items

- **Objective:** Identify best-selling products over a custom timeframe (default 30 days).
- **Calculation (SQL):** Joins `invoice_items` with `sales_invoices`. `Sum(quantity)` grouped by `product_name`.
- **Display:** Shows a data table sorted in descending order by `total_sold`.

7.2 Slow Moving Items

- **Objective:** Identify underperforming products.
- **Calculation (SQL):** Filters `stock_items` where `ItemQuantity > 0`, joins with `invoice_items` and `sales_invoices`. Sums the quantity sold. It explicitly targets items where total sales are low (e.g., `< 5` units).
- **Display:** Shows a data table sorted in ascending order.

7.3 Dead Stock (Unused Items)

- **Objective:** Identify tied-up capital in inventory that hasn't sold in the last 180 days.
- **Calculation (SQL):** Filters `stock_items` where `ItemQuantity > 0` but the `ItemName` does not exist in any `sales_invoices` for the specified period.
- **Dead Stock Value:** `Sum(ItemQuantity * ItemRate)` for all unused items.

7.4 Dormant Customers (Unused Ledgers)

- **Objective:** Identify customers who haven't made a purchase recently to target re-engagement.
- **Calculation (SQL):** Filters the `customers` table for active customers whose `customer_name` is absent from `sales_invoices` over the last 180 days.
- **Display:** The KPI count evaluates the length of the returned array (`list.length`), and the data is rendered in a tabular view.

7.5 Daily Sales & Estimated Gross Profit

- **Objective:** Visualize a 14-day trend of revenue vs. profit.
- **Revenue Calculation:** `Sum(total_amount)` of all invoices per day.
- **Profit Calculation (SQL):** `Sum((unit_price - COALESCE(StandardCost, ItemRate, 0)) * quantity)`. It defaults to the `StandardCost`, falls back to `ItemRate`, and uses `0` if neither exists.
- **Display:** Rendered using a dual-bar chart (`fl_chart`). The Y-axis (`maxY`) automatically scales based on the highest revenue recorded (`maxRev > 0 ? maxRev * 1.2 : 100`) to ensure stability.